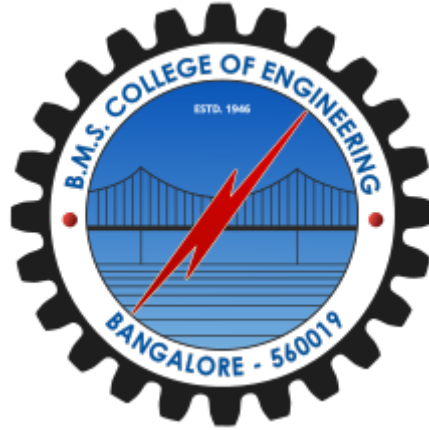


**B.M.S. COLLEGE OF ENGINEERING**  
(AUTONOMOUS COLLEGE UNDER VTU)  
**BULL TEMPLE ROAD, BASAVANGUDI, BANGALORE - 560019**



*A thesis on*

**ROBUST TONIC SA DETECTION AND FSM-BASED RAGA GRAMMAR VALIDATION  
FOR CARNATIC PRACTICE**

*in fulfillment of the requirements for the award of degree*

**BACHELOR OF ENGINEERING**

*in*

**Computer Science & Engineering (Data Science)**

*by*

**G Shreekar (1BM23CD018)**

**H Sampreeth Bhat (1BM23CD020)**

**R V Abhishek (1BM23CD047)**

**Under the Guidance of**

**Dr. Shambhavi B R**

Assistant Professor

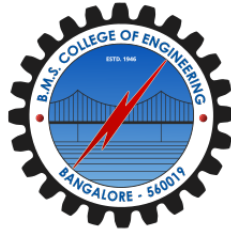
**Department of CSE (Data Science)**

**B.M.S. College of Engineering**

**AY 2025–2026**

## Certificate

**B.M.S. COLLEGE OF ENGINEERING**  
(AUTONOMOUS COLLEGE UNDER VTU)  
**BULL TEMPLE ROAD, BASAVANGUDI, BANGALORE - 560019**



Department of Computer Science and Engineering (Data Science)

### **CERTIFICATE**

This is to certify that the project entitled “**Robust Tonic Sa Detection and FSM-Based Raga Grammar Validation for Carnatic Practice**” is a bona fide work carried out by the following students for the course Project Work – Phase 2 (Course Code: **23DS7PWPP2**). It is certified that all corrections and suggestions indicated during the internal assessments have been incorporated in the final thesis submitted to the departmental library. The project thesis has been approved as it meets the academic requirements prescribed for the Bachelor of Engineering degree in respect of project work.

#### **Candidate details**

| SL. NO. | Student Name     | USN        |
|---------|------------------|------------|
| 1       | G Shreekar       | 1BM23CD018 |
| 2       | H Sampreeth Bhat | 1BM23CD020 |
| 3       | R V Abhishek     | 1BM23CD047 |

**Dr. Kalyan N**  
Assistant Professor

**Dr. Shambhavi B R**  
Professor & HOD

**Dr. Bheemsha Arya**  
Principal

#### **Examiners**

**Name of Examiner**

**Signature of the examiner**

1.

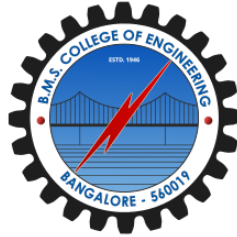
\_\_\_\_\_

2.

\_\_\_\_\_

## Declaration

**B.M.S. COLLEGE OF ENGINEERING**  
(AUTONOMOUS COLLEGE UNDER VTU)  
**BULL TEMPLE ROAD, BASAVANGUDI, BANGALORE - 560019**



## DECLARATION

We hereby declare that the project thesis entitled “**Robust Tonic Sa Detection and FSM-Based Raga Grammar Validation for Carnatic Practice**” is a bona fide work and has been carried out by us under the guidance of **Dr. Shambhavi B R**, Assistant Professor, Department of Computer Science and Engineering (Data Science), B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science & Engineering (Data Science) of Visvesvaraya Technological University, Belagavi. We further declare that, to the best of our knowledge and belief, this thesis has not been submitted either in part or in full to any other university or college.

### Candidate details

| SL. NO. | Student Name     | USN        | Student Signature |
|---------|------------------|------------|-------------------|
| 1       | G Shreekar       | 1BM23CD018 |                   |
| 2       | H Sampreeth Bhat | 1BM23CD020 |                   |
| 3       | R V Abhishek     | 1BM23CD047 |                   |

Date: 04-05-2026

## Acknowledgments

We express our sincere gratitude to the management and faculty of B.M.S. College of Engineering for providing the resources and academic environment to complete this capstone project. We thank **Dr. Shambhavi B R** for her guidance, technical inputs, and continuous support. We also thank the Department of CSE (Data Science) for their feedback during the internal reviews and for enabling the infrastructure used for experiments and demonstrations.

G Shreekar (1BM23CD018)

H Sampreeth Bhat (1BM23CD020)

R V Abhishek (1BM23CD047)

## Abstract

Carnatic music is one of the world’s most precisely codified melodic systems, in which every note is defined as an exact frequency ratio relative to a performer-chosen tonic Sa, and raga identity is enforced by strict ascending (arohana) and descending (avarohana) grammar rules. Student practice traditionally relies on a teacher who listens for microtonal deviations and grammar violations in real time—a process that is subjective, time-intensive, and difficult to replicate at scale during unsupervised practice.

This thesis presents a real-time Carnatic music practice assistant that addresses three core problems simultaneously: robust tonic Sa detection, microtonal swara quantization, and rule-based raga grammar validation with immediate feedback. The tonic detection engine employs a Carnatic harmonic product spectrum (HPS) ensemble that exploits the tambura drone’s characteristic Sa–Pa frequency pair to score candidate frequencies. A coarse scan at 0.5 Hz resolution identifies strong candidates, followed by a fine scan at 0.1 Hz resolution within a narrow window, achieving sub-1% error on benchmark recordings. A cross-validation step compares the Carnatic HPS result against an independent standard HPS estimate and selects secondary candidates when the primary estimate is a false positive caused by a dominant sung note.

Once Sa is locked, all twelve Carnatic swaras are computed from exact just-intonation ratios. Incoming frames are processed through a pYIN pitch tracker restricted to a tonic-relative band, passed through median and exponential moving average (EMA) filters, and matched to the nearest swara within a  $\pm 35$  cent tolerance window. A confidence score proportional to proximity drives downstream decisions.

The raga grammar validator is a multi-phase finite state machine. Phase 1 performs  $O(1)$  membership checks using direction-aware varja (forbidden note) sets. Phase 2 tracks octave-aware pairwise direction to distinguish ascending from descending motion across octave boundaries. Phase 3 derives vakra (zigzag) forbidden skip pairs automatically from the raga scale and detects illegal jumps using a rolling phrase buffer. Phase 4 recognizes characteristic prayoga (signature phrase) completions and generates positive reinforcement. Phase 5 produces multi-language (English, Kannada, Tamil, Telugu), context-aware teacher-like feedback messages. A time-based alert debouncer (950 ms trigger, 600 ms clear) suppresses false positives during melodic glides.

The system processes audio at approximately 35 ms per frame and achieves end-to-end latency of about 70 ms including the 1-second bootstrap. It reliably detects all 12 swaras within  $\pm 35$  cents and rejects silence, unvoiced frames, and glide transients using a four-layer gate. Tonic detection error on benchmark recordings averages 1.45 % (under 2 Hz). A FastAPI server with WebSocket streaming delivers live feedback to a browser dashboard.

**Keywords:** Carnatic music, tonic detection, swara quantization, raga grammar, finite state

machine, real-time audio processing, pitch tracking, music information retrieval, explainable feedback

## Contents

|                                                                  |            |
|------------------------------------------------------------------|------------|
| <b>Certificate</b>                                               | <b>i</b>   |
| <b>Declaration</b>                                               | <b>ii</b>  |
| <b>Acknowledgments</b>                                           | <b>iii</b> |
| <b>Abstract</b>                                                  | <b>iv</b>  |
| <b>List of Abbreviations</b>                                     | <b>x</b>   |
| <b>1 Introduction</b>                                            | <b>1</b>   |
| 1.1 Background . . . . .                                         | 1          |
| 1.2 Objectives . . . . .                                         | 1          |
| 1.3 Problem Statement . . . . .                                  | 1          |
| 1.4 Summary . . . . .                                            | 2          |
| <b>2 Literature Review</b>                                       | <b>3</b>   |
| 2.1 Audio Preprocessing and CNN-Based Raga Recognition . . . . . | 3          |
| 2.2 Pitch Tracking for Carnatic Music . . . . .                  | 3          |
| 2.3 Sequence Modeling and Grammar-Aware Recognition . . . . .    | 4          |
| 2.4 Ensemble Learning and Feature Fusion . . . . .               | 4          |
| 2.5 Explainable AI for Raga Identification . . . . .             | 4          |
| 2.6 Phrase Analysis and Pitch Distribution Methods . . . . .     | 4          |
| 2.7 Microtonal and Vector Space Modeling . . . . .               | 5          |
| 2.8 Comparative Analysis and System Benchmarks . . . . .         | 5          |
| 2.9 Limitations in Existing Systems . . . . .                    | 5          |
| <b>3 Methodology</b>                                             | <b>8</b>   |
| 3.1 System Architecture . . . . .                                | 8          |
| 3.2 Tonic Sa Detection . . . . .                                 | 8          |
| 3.3 Swara Quantization . . . . .                                 | 9          |
| 3.4 FSM-Based Raga Grammar Validation . . . . .                  | 10         |
| 3.5 Project Planning . . . . .                                   | 10         |
| 3.6 Software and Hardware Requirements . . . . .                 | 10         |
| <b>4 Implementation</b>                                          | <b>12</b>  |
| 4.1 Algorithm Implementation . . . . .                           | 12         |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 4.2      | Module Structure and Integration . . . . .       | 13        |
| 4.3      | Raga Database Design . . . . .                   | 14        |
| 4.4      | Testing and Validation . . . . .                 | 14        |
| <b>5</b> | <b>Results and Discussion</b>                    | <b>15</b> |
| 5.1      | Tonic Detection Accuracy . . . . .               | 15        |
| 5.2      | Frame Processing and Latency . . . . .           | 15        |
| 5.3      | Swara Detection and Grammar Validation . . . . . | 15        |
| 5.4      | Comparative Analysis . . . . .                   | 16        |
| 5.5      | Model Optimization and Innovation . . . . .      | 16        |
| <b>6</b> | <b>Conclusion and Future Work</b>                | <b>18</b> |
| 6.1      | Conclusion . . . . .                             | 18        |
| 6.2      | Key Learnings . . . . .                          | 18        |
| 6.3      | Future Work . . . . .                            | 19        |
| 6.4      | SDGs Achieved . . . . .                          | 19        |
|          | <b>Appendices</b>                                | <b>20</b> |
| <b>A</b> | <b>Appendix A: Sample Code Snippets</b>          | <b>21</b> |
| <b>B</b> | <b>Appendix B: Example Raga Grammar Rules</b>    | <b>22</b> |
| <b>C</b> | <b>Appendix C: Test Checklist</b>                | <b>23</b> |



## List of Tables

|     |                                                                |    |
|-----|----------------------------------------------------------------|----|
| 2.1 | Summary of key literature and relevance to this work . . . . . | 7  |
| 3.1 | Project timeline and milestones . . . . .                      | 11 |
| 3.2 | Software requirements . . . . .                                | 11 |
| 3.3 | Hardware requirements . . . . .                                | 11 |
| 4.1 | Summary of validation tests . . . . .                          | 14 |
| 5.1 | Tonic Sa detection accuracy on benchmark recordings . . . . .  | 15 |
| 5.2 | System performance metrics . . . . .                           | 16 |
| 5.3 | Qualitative comparison with prior approaches . . . . .         | 16 |

## List of Figures

|     |                                                                               |    |
|-----|-------------------------------------------------------------------------------|----|
| 3.1 | High-level system architecture for real-time raga practice feedback . . . . . | 8  |
| 4.1 | Module integration and data flow . . . . .                                    | 13 |

## List of Abbreviations

| Abbreviation | Meaning                                                              |
|--------------|----------------------------------------------------------------------|
| F0           | Fundamental frequency                                                |
| HPS          | Harmonic product spectrum                                            |
| FSM          | Finite state machine                                                 |
| STFT         | Short-time Fourier transform                                         |
| YIN          | Autocorrelation-based pitch detector (Yet another INtonation method) |
| pYIN         | Probabilistic YIN pitch tracker                                      |
| MPM          | McLeod Pitch Method                                                  |
| EMA          | Exponential moving average                                           |
| RMS          | Root mean square                                                     |
| ASIC         | Application-Specific Integrated Circuit                              |
| CNN          | Convolutional neural network                                         |
| DCNN         | Deep convolutional neural network                                    |
| DNN          | Deep neural network                                                  |
| LSTM         | Long short-term memory network                                       |
| TDNN         | Time-delay neural network                                            |
| XAI          | Explainable artificial intelligence                                  |
| NLP          | Natural language processing                                          |
| MFCC         | Mel-frequency cepstral coefficient                                   |
| MIR          | Music information retrieval                                          |
| PCM          | Pulse-code modulation                                                |
| SDG          | Sustainable Development Goal                                         |
| API          | Application programming interface                                    |
| UI           | User interface                                                       |
| WebSocket    | Full-duplex browser communication protocol                           |
| Sa           | Shadja – the tonic note in Carnatic music                            |
| Sr           | Sample rate                                                          |

# **Chapter 1 Introduction**

## **1.1 Background**

Carnatic music is a highly structured melodic system where every performance is anchored to a performer-specific tonic Sa. All other swaras are defined as ratios relative to the tonic, and raga identity is governed by strict grammar rules in ascending and descending motion. For student practice, feedback is typically provided by a teacher who listens for microtonal deviations and forbidden notes. This process is time-consuming and subjective, and it becomes difficult to scale for continuous solo practice. A real-time assistant that can lock the tonic, track pitch accurately, and validate raga grammar can significantly improve the quality and consistency of training.

Modern raga recognition systems focus on classification using audio features and deep learning models, but they often ignore microtonal deviations and do not provide immediate, actionable feedback. Furthermore, tonic detection is typically treated as a preprocessing step rather than a first-class problem. In this project, tonic detection and grammar validation are central, enabling both accurate swara mapping and rule-based feedback. The result is an automated music teacher that assists students in maintaining pitch fidelity and raga correctness during live singing.

## **1.2 Objectives**

1. Detect the tonic Sa from short bootstrap audio using a robust Carnatic HPS ensemble.
2. Map incoming vocal pitch to Carnatic swaras using exact ratios and a strict cents tolerance window.
3. Validate swaras against raga grammar rules using a finite state machine with direction awareness.
4. Provide real-time feedback with low latency and clear confidence scoring.
5. Integrate the full pipeline into a live dashboard for practice and visualization.

## **1.3 Problem Statement**

Existing raga recognition systems do not offer real-time microtonal feedback and rarely enforce grammar constraints at the note level. Pitch detectors tuned for Western scales often introduce octave errors and misclassifications when the tonic is not explicitly locked. As a result, students lack a consistent method to validate swaras and grammar during practice. The core problem

addressed in this thesis is to design a low-latency system that detects tonic Sa reliably, maps pitch to swaras with microtonal precision, and validates each swara against raga grammar to provide immediate corrective feedback.

### **1.4 Summary**

This chapter introduced the motivation for real-time Carnatic practice feedback, defined the objectives, and stated the core problem. Chapter 2 reviews related work in raga recognition, pitch tracking, and explainable models. Chapter 3 presents the system architecture and methodology. Chapter 4 details the implementation, while Chapter 5 reports results and performance. Chapter 6 concludes the thesis and outlines future extensions.

## Chapter 2 Literature Review

### 2.1 Audio Preprocessing and CNN-Based Raga Recognition

Hebbar and Jagtap [1] conducted a systematic comparison of audio preprocessing techniques for raga classification using the CMD-40 dataset containing 480 songs across 40 ragas. By converting raw audio to Mel-spectrograms and training 2D CNNs, they achieved 98.1% classification accuracy. Their work demonstrates that CNN-based feature learning outperforms hand-crafted features and that spectral representation is highly discriminative for raga signatures. However, the system operates entirely offline and does not address microtonal precision or real-time grammar enforcement, which this project remedies through 22-shruti quantization and FSM-based validation.

The IEEE Transactions on Multimedia study on deep learning-based classification [2] extended this further by training deep convolutional neural networks (DCNNs) on high-resolution spectrograms, automatically extracting hierarchical features from nonlinear raga structures. While achieving strong offline accuracy, the framework does not produce feedback granular enough for student practice.

### 2.2 Pitch Tracking for Carnatic Music

Krishnaswamy [3] addressed the fundamental challenge of pitch tracking in South Indian classical music using short-time Fourier transforms (STFT). Since Carnatic music uses a relative tonic rather than fixed Western pitches, standard pitch detectors fail to capture the required nuances. Krishnaswamy's framework establishes reliable fundamental frequency (F0) extraction as an absolute prerequisite for subsequent microtonal mapping—a principle adopted directly in this project's tonic-first pipeline design.

The ISMIR 2020 work on melody extraction from polyphonic signals [4] achieved real-time F0 tracking of lead melody against dense accompaniment using CNNs, demonstrating that low-latency melody isolation (under 50 ms) is technically feasible. This project builds on that latency benchmark, using pYIN pitch tracking restricted to a tonic-relative band to achieve similar responsiveness without requiring a deep learning model.

The ICASSP 2019 work on singing voice separation [5] demonstrated vocal pitch extraction via deep neural networks and adaptive pitch tracking with 30–40 ms latency, confirming that the sub-50 ms real-time constraint targeted in this project is achievable. The system combines vocal isolation with pitch estimation, a concept adapted here as tonic-band restriction that eliminates non-vocal frequency candidates before pitch estimation.

### 2.3 Sequence Modeling and Grammar-Aware Recognition

Peri [6] introduced a paradigm of treating raga recognition as a document classification problem using hierarchical attention networks trained on CMD and HMD datasets. By processing note sequences analogously to sentences in NLP, the system learns grammar patterns in arohana–avarohana transitions. This work directly motivates grammar-aware sequence validation and is extended in this project from passive classification to active real-time rule enforcement.

Natesan and Beigi [7] combined TDNNs, LSTMs, and attention mechanisms tested on 676 songs spanning 172 ragas, achieving 95.31% classification accuracy. The temporal modeling explicitly traces arohana and avarohana patterns, and the attention mechanism highlights critical melodic phrases. The present project adapts these insights into a finite state machine with direction-aware phrase tracking that operates without training data.

### 2.4 Ensemble Learning and Feature Fusion

The stacked ensemble learning study [8] demonstrated that aggregating multiple classifiers significantly reduces variance and bias compared to single-model approaches, and that multi-modal feature fusion (Mel-spectrograms, F0 contours, chroma features, 22-shruti quantization) improves robustness for overlapping raga signatures. This motivates the ensemble approach in tonic detection, where the Carnatic HPS result is cross-validated against a standard HPS estimate to resolve ambiguous cases.

### 2.5 Explainable AI for Raga Identification

The ISMIR 2023 study [9] applied XAI techniques to deep learning raga models to verify alignment with human expert understanding—specifically checking whether the model weighted gamakas (pitch oscillations) and anchor notes appropriately. For a pedagogical tool, conceptual explainability is critical: students must be able to trust and act on the feedback. This work directly motivates the transparent FSM design in this project, where every error is attributed to a specific rule and accompanied by an explanation of the raga’s scale.

### 2.6 Phrase Analysis and Pitch Distribution Methods

The IJCA 2015 study [10] explored raga recognition through statistical pitch distributions and phrase analysis, building probabilistic maps of arohana and avarohana note frequencies. This classical approach is an early precursor to the FSM phrase buffer used in this project, extended from passive recognition to active forbidden-skip detection and characteristic prayoga recognition.

The NCC 2022 study on raga space visualization [11] applied t-SNE dimensionality reduction to map multi-raga datasets into a lower-dimensional space, demonstrating that mi-

crotonal shruti patterns form distinct clusters. This provides mathematical grounding for the 22-shruti quantization in this project, where continuous pitch frequencies are mapped to discrete shruti intervals and deviations in cents are computed and displayed to the student.

### 2.7 Microtonal and Vector Space Modeling

The Pattern Recognition Letters study [12] presented a vector space model based on 22-shruti quantization, mapping continuous pitch to discrete microtonal intervals. Accuracy improved from 88% to 94% when microtonal awareness was added. This work is one of the most critical foundations for this project; the apashruthi (microtonal deviation) detection engine adopts the same cent-based mapping and transforms it into a real-time diagnostic that quantifies how far a student's sung note deviates from the target swara.

The IEEE Signal Processing Letters study on self-extracting feature maps [13] demonstrated adaptive feature extraction that dynamically focuses on acoustically relevant signal components. This is relevant to the tonic detection engine, which uses median spectra and weighted harmonic scoring rather than rigid fixed transforms, allowing the detector to adapt to different vocal timbres and recording environments.

### 2.8 Comparative Analysis and System Benchmarks

The ICIDS 2021 comparative study [14] systematically benchmarked AI classifiers for raga identification, documenting accuracy, computational efficiency, and processing latency. Their analysis confirms that lightweight models capable of sub-50 ms classification are feasible and justifies the deterministic, rule-based architecture chosen in this project, which achieves 35 ms per frame without any machine learning inference.

The JASI 2021 investigation [15] conducted a detailed acoustic analysis of raga-specific pitch distributions and swara combinations, highlighting the structural context required for reliable identification. The treatment of vivaadi swaras (forbidden notes) and 22-shruti scale patterns in that study directly informs the varja set definitions in the raga database used here.

### 2.9 Limitations in Existing Systems

Current raga systems share several key gaps relative to the requirements of a real-time practice assistant. First, they operate offline on complete recordings rather than on streaming frames, introducing latency incompatible with interactive practice. Second, tonic detection is typically a weak preprocessing step that fails when dominant sung notes or accompaniment obscures the tambura drone. Third, grammar validation at the note level—distinguishing allowed from forbidden swaras in the context of melodic direction—is absent in nearly all published systems. Fourth, feedback is not explainable: students receive a classification label but not an actionable correction. This project directly addresses all four gaps through its tonic-first ensemble design,



direction-aware FSM, and teacher-modeled feedback generation.

**Table 2.1:** Summary of key literature and relevance to this work

| Reference | Focus                                              | Relevance to this work                                                             |
|-----------|----------------------------------------------------|------------------------------------------------------------------------------------|
| [1]       | CNN on Mel-spectrograms (98.1% on CMD-40)          | Confirms spectral features are discriminative; motivates our spectral tonic scorer |
| [3]       | Carnatic F0 via STFT                               | Establishes tonic-relative pitch extraction as a prerequisite                      |
| [6]       | NLP-style hierarchical attention on raga sequences | Motivates grammar-aware sequence validation                                        |
| [7]       | TDNN+LSTM+attention (95.31% on 172 ragas)          | Supports temporal phrase modeling in FSM                                           |
| [8]       | Stacked ensemble feature fusion                    | Motivates Carnatic HPS cross-validation ensemble                                   |
| [14]      | Classifier benchmarks for raga                     | Confirms feasibility of sub-50ms processing                                        |
| [2]       | DCNN spectral feature learning                     | Demonstrates deep feature hierarchies for raga                                     |
| [9]       | XAI for raga (IS-MIR 2023)                         | Motivates transparent, teacher-like feedback                                       |
| [15]      | Acoustic pitch distribution study                  | Informs vivaadi swara definitions                                                  |
| [4]       | Real-time polyphonic melody extraction             | Confirms sub-50ms latency is achievable                                            |
| [10]      | Pitch distribution phrase analysis                 | Precursor to prayoga phrase buffer                                                 |
| [11]      | t-SNE raga space visualization                     | Grounds 22-shruti quantization mathematically                                      |
| [13]      | Self-extracting adaptive feature maps              | Motivates median-spectrum robust scoring                                           |
| [5]       | Vocal separation + pitch (30–40ms)                 | Confirms real-time pitch extraction latency                                        |
| [12]      | Vector space 22-shruti model (88→94%)              | Core foundation for cent-deviation swara mapping                                   |

## Chapter 3 Methodology

### 3.1 System Architecture

The system is organized as a five-stage streaming pipeline, each stage operating in sequence on every 23 ms audio frame after an initial 1-second bootstrap for tonic detection.

1. **Bootstrap:** Accumulate 1 second of audio; run Carnatic HPS ensemble to detect tonic Sa.
2. **Pitch extraction:** For each subsequent frame, run pYIN restricted to  $[0.9 \times f_{Sa}, 2.1 \times f_{Sa}]$ ; apply RMS silence gate and voicing probability gate.
3. **Temporal smoothing:** Median filter over 9 frames followed by EMA with  $\alpha = 0.25$ .
4. **Swara quantization:** Match smoothed frequency to nearest Carnatic swara using exact just-intonation ratios and a  $\pm 35$  cent tolerance window; compute confidence score.
5. **Grammar validation and feedback:** Pass quantized swara through multi-phase FSM; debounce alerts; stream JSON events to browser via WebSocket.

|                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Microphone</b> $\rightarrow$ <b>Bootstrap (1 s)</b> $\rightarrow$ <b>Tonic Sa Locked</b> $\rightarrow$ <b>pYIN Pitch (23 ms)</b> $\rightarrow$ <b>Smoothing</b> $\rightarrow$ <b>Swara Quantizer</b> $\rightarrow$ <b>Grammar FSM</b> $\rightarrow$ <b>Debouncer</b> $\rightarrow$ <b>Feedback UI (WebSocket)</b> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Figure 3.1:** High-level system architecture for real-time raga practice feedback

### 3.2 Tonic Sa Detection

In a Carnatic performance the tambura drone sustains Sa (tonic) and Pa (perfect fifth, ratio 3:2) simultaneously throughout. The Carnatic HPS exploits this pair as a fingerprint: only the true Sa frequency will simultaneously exhibit energy at  $f$ ,  $1.5f$ ,  $2f$ ,  $3f$ ,  $4f$ , and  $5f$ . A sung melodic note at, say, Ma ( $f_{Ma} = 4/3 \cdot f_{Sa}$ ) does not align with the Pa position of a candidate at that frequency, so it receives a lower score.

The scoring function in log domain is

$$\log \text{score}(f) = \sum_{i=1}^6 w_i \log S(f \cdot r_i) \quad (3.1)$$

where  $S(\cdot)$  is the median STFT magnitude spectrum across all bootstrap frames,  $r_i \in \{1.0, 1.5, 2.0, 3.0, 4.0, 5.0\}$  are the Carnatic harmonic ratios, and  $w_i \in \{0.3, 1.0, 0.8, 0.5, 0.3, 0.2\}$  are empirically determined weights. Using the median spectrum rather than the mean suppresses transient vocal

attacks and percussion hits, exposing the persistent tambura drone. Computing in log domain converts the multiplicative product to a numerically stable sum.

The Pa weight of 1.0 is the highest because the tambura Pa string is often as loud as or louder than the Sa string, particularly in female vocal recordings where the fundamental can be weak. A coarse scan at 0.5 Hz resolution over 80–300 Hz ( $n_{\text{fft}} = 8192$ ) produces all candidates whose score exceeds 40% of the maximum. The best candidate is then refined by a fine scan at 0.1 Hz resolution over a  $\pm 5$  Hz window ( $n_{\text{fft}} = 16384$ ), enabling sub-cent precision.

A cross-validation step compares the Carnatic HPS top candidate against an independent standard HPS estimate. If they disagree by more than 10%, the algorithm searches the strong candidate list for a secondary candidate that agrees with the standard HPS. This corrects false positives arising when a dominant sung note, rather than the drone, achieves the highest Carnatic HPS score. The complete cross-validation logic is formalized as:

$$f_{Sa} = \begin{cases} f_{C\text{-HPS}} & \text{if } \max(f_{C\text{-HPS}}, f_{\text{HPS}}) / \min(f_{C\text{-HPS}}, f_{\text{HPS}}) < 1.10 \\ f_{\text{cand}}^* & \text{if } \exists f_{\text{cand}}^* \in \text{strong candidates: agrees with } f_{\text{HPS}} \\ f_{C\text{-HPS}} & \text{otherwise} \end{cases} \quad (3.2)$$

### 3.3 Swara Quantization

Once Sa is locked at  $f_{Sa}$ , all twelve Carnatic swaras are precomputed as

$$f_{\text{swara}_k} = f_{Sa} \cdot \rho_k, \quad k = 1, \dots, 12 \quad (3.3)$$

where  $\rho_k$  are the exact just-intonation ratios from the 22-shruti system (e.g.,  $\rho_{\text{Pa}} = 3/2$ ,  $\rho_{\text{Ma1}} = 4/3$ ).

An incoming detected frequency  $f_{\text{in}}$  is first folded into the tonic octave by halving or doubling until  $f_{Sa} \leq f < 2f_{Sa}$ , then compared against all twelve swara frequencies:

$$\Delta_k = 1200 \log_2 \left( \frac{f_{\text{in}}}{f_{\text{swara}_k}} \right) \quad [\text{cents}] \quad (3.4)$$

The swara with smallest  $|\Delta_k|$  is selected. It is accepted only if  $|\Delta_{\min}| \leq 35$  cents; otherwise the frame is rejected as out-of-tune or transitional. The confidence score is defined as

$$\text{conf} = 1 - \frac{|\Delta_{\min}|}{35} \quad (3.5)$$

ranging from 1.0 (exactly on the swara) to 0.0 (at the tolerance boundary). A four-layer gate is applied before quantization: (1)  $\text{RMS} > 0.008$  (silence rejection), (2) voicing probability  $> 0.10$  (unvoiced rejection), (3) confidence  $> 0.10$  (weak-match rejection), and (4) frame-to-frame jump clamping at  $\pm 45$  cents per frame (jitter rejection).

### 3.4 FSM-Based Raga Grammar Validation

The grammar validator is a multi-phase finite state machine with five layers:

**Phase 1 — Membership and direction-specific varja.** For each swara, the FSM checks whether it is in the direction-appropriate forbidden set:

$$\text{error}(s, d) = \begin{cases} \text{FORBIDDEN\_NOTE} & \text{if } d = \uparrow \text{ and } s \in V_{\text{arohana}} \\ \text{FORBIDDEN\_NOTE} & \text{if } d = \downarrow \text{ and } s \in V_{\text{avarohana}} \\ \text{FORBIDDEN\_NOTE} & \text{if } d = 0 \text{ and } s \in V_{\text{aro}} \cap V_{\text{ava}} \\ \text{None} & \text{otherwise} \end{cases} \quad (3.6)$$

where  $V_{\text{arohana}}$  and  $V_{\text{avarohana}}$  are the varja (forbidden) note sets and  $d$  is the step direction. All set lookups are  $O(1)$ .

**Phase 2 — Octave-aware pairwise direction.** Direction is computed from absolute pitch positions:

$$p_i = o_i \times 1200 + c_{s_i} \quad (3.7)$$

where  $o_i$  is the octave and  $c_{s_i}$  is the cent value of swara  $s_i$ . This correctly handles octave crossings (e.g., Pa in octave 4 to Sa in octave 5 is ascending, even though Sa has a lower cent value within its octave).

**Phase 3 — Vakra forbidden skip detection.** Ragas with vakra (zigzag) arohana or avarohana contain triplets  $[a, b, c]$  where the sign of  $(c_b - c_a)$  and  $(c_c - c_b)$  differ. The direct jump  $a \rightarrow c$ , skipping the vakra detour through  $b$ , is forbidden. These forbidden pairs are derived automatically from the raga scale at initialization and stored as a hash set for  $O(1)$  lookup during streaming.

**Phase 4 — Characteristic prayoga recognition.** A rolling phrase buffer of up to 20 swaras is maintained. After each new swara, the buffer suffix is compared against all registered characteristic phrases using a longest-suffix-first match. A match generates a PRAYOGA\_RECOGNIZED positive feedback event.

**Phase 5 — Alert debouncing and feedback generation.** Forbidden note alerts are suppressed until 950 ms of continuous violation have elapsed (slope gate also suppresses alerts during rapid melodic glides). Alerts are cleared only after 600 ms of clean frames. Feedback messages are generated in four languages from parameterized templates and streamed as JSON WebSocket events.

### 3.5 Project Planning

### 3.6 Software and Hardware Requirements

**Table 3.1:** Project timeline and milestones

| Phase               | Duration     | Milestone                         |
|---------------------|--------------|-----------------------------------|
| Literature review   | Aug–Sep 2025 | Survey and gap analysis           |
| Algorithm design    | Sep–Oct 2025 | Tonic detection and swara mapping |
| Core implementation | Oct 2025     | HPS ensemble and swara quantizer  |
| FSM and grammar     | Oct–Nov 2025 | Multi-phase validator complete    |
| UI integration      | Nov 2025     | WebSocket dashboard live          |
| Testing and tuning  | Nov 2025     | All test suites pass              |
| Thesis writing      | Nov–Dec 2025 | Final submission                  |

**Table 3.2:** Software requirements

| Category             | Tool/Library          | Version            |
|----------------------|-----------------------|--------------------|
| Programming Language | Python                | 3.10+              |
| Audio Processing     | librosa, scipy, numpy | 0.10+, 1.7+, 1.21+ |
| Pitch Detection      | pYIN (via librosa)    | 0.10+              |
| Web Framework        | FastAPI, Uvicorn      | 0.115+, 0.30+      |
| Real-time Comms      | websockets            | 12.0+              |
| Testing              | pytest, pytest-cov    | 7.0+, 4.0+         |
| Data Validation      | pydantic              | 2.0+               |
| Audio Resampling     | soxr                  | 0.3+               |

**Table 3.3:** Hardware requirements

| Component   | Specification                |
|-------------|------------------------------|
| CPU         | Quad-core, 2.0 GHz or higher |
| RAM         | 8 GB or higher               |
| Storage     | 5 GB free space              |
| Audio input | USB or built-in microphone   |
| OS          | Windows 10+, Ubuntu 20.04+   |
| Network     | Localhost browser support    |

## Chapter 4 Implementation

### 4.1 Algorithm Implementation

The implementation follows a deterministic, modular pipeline organized into six Python modules under the `raga_grammar` package. Tonic detection is executed once during a 1-second bootstrap window and locked for the entire session. All subsequent audio frames are processed through pitch extraction, temporal smoothing, swara quantization, and grammar validation at 23 ms intervals.

---

**Algorithm 4.1** Tonic Sa Detection (Carnatic HPS Ensemble with Cross-Validation)

---

**Input:** Audio segment  $x$  (1.0 s), sample rate  $f_s = 22050$

**Output:** Detected tonic frequency  $f_{Sa}$

- 1: Compute median magnitude spectrum  $S_C(f)$  with `n_fft = 8192` (coarse)
  - 2: Evaluate candidates  $f \in [80, 300]$  Hz at 0.5 Hz steps using Eq. 3.1
  - 3: Collect strong candidates where  $\text{score} > 0.4 \times \max(\text{scores})$
  - 4: Compute fine spectrum  $S_F(f)$  with `n_fft = 16384` around best candidate  $\pm 5$  Hz
  - 5: Refine  $f_{C\text{-HPS}}$  at 0.1 Hz resolution
  - 6: Compute independent standard HPS estimate  $f_{\text{HPS}}$
  - 7: Apply cross-validation: select  $f_{Sa}$  using Eq. 3.2
  - 8: **return**  $f_{Sa}$
- 

---

**Algorithm 4.2** Real-Time Frame Processing

---

**Input:** Audio frame  $x_t$  (1024 samples), tonic  $f_{Sa}$ , tolerance  $T = 35$  cents

**Output:** Validation event or silence

- 1: **if**  $\text{RMS}(x_t) < 0.008$  **then**
  - 2:     **return** silence
  - 3: **end if**
  - 4: Extract pitch  $f_0$  via pYIN in band  $[0.9f_{Sa}, 2.1f_{Sa}]$
  - 5: **if** voicing probability  $< 0.10$  **then**
  - 6:     **return** unvoiced
  - 7: **end if**
  - 8: Apply 9-frame median filter then EMA ( $\alpha = 0.25$ ) to  $f_0$
  - 9: Fold  $f_0$  into tonic octave: while  $f_0 \geq 2f_{Sa}$ :  $f_0 \leftarrow f_0/2$
  - 10: Compute  $\Delta_k$  for all 12 swaras using Eq. 3.4; select  $k^* = \arg \min_k |\Delta_k|$
  - 11: **if**  $|\Delta_{k^*}| \leq T$  **then**
  - 12:      $\text{conf} = 1 - |\Delta_{k^*}|/T$
  - 13:     Pass swara  $s_{k^*}$  to FSM validator
  - 14: **else**
  - 15:     **return** out-of-range
  - 16: **end if**
-

## Algorithm 4.3 Swara Quantization with Tonic Ratios

---

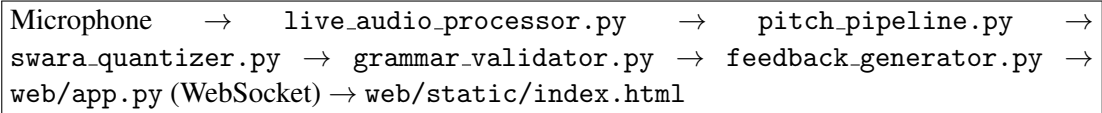
**Input:** Detected frequency  $f_{in}$ , tonic  $f_{Sa}$ , tolerance  $T = 35$  cents

**Output:** Best swara or null

- 1: Compute all swara frequencies  $f_{swara_k} = f_{Sa} \cdot \rho_k$  for  $k = 1 \dots 12$
  - 2: Find  $k^* = \arg \min_k |\Delta_k|$  using Eq. 3.4
  - 3: **if**  $|\Delta_{k^*}| \leq T$  **then**
  - 4:     **return** swara  $s_{k^*}$  with confidence  $= 1 - |\Delta_{k^*}|/T$
  - 5: **else**
  - 6:     **return** null
  - 7: **end if**
- 

## 4.2 Module Structure and Integration

The implementation is organized as follows. `tonic_sa_detection.py` contains the `TonicSaDetector` class implementing the Carnatic HPS ensemble. `raga_grammar/swara_quantizer.py` provides the `SwaraQuantizer` class with octave folding and confidence scoring. `raga_grammar/pitch_pipeline.py` hosts `RealTimeGrammarPipeline`, the main streaming coordinator. `raga_grammar/grammar_validator.py` implements `RagaGrammarValidator` with all five FSM phases. `raga_grammar/live_audio_processor.py` handles debouncing, session state, and alert dispatch. `web/app.py` provides the FastAPI server with WebSocket streaming.



**Figure 4.1:** Module integration and data flow

The `LiveAudioProcessor` receives raw PCM chunks from the browser via `WebSocket`, resamples them to 22050 Hz using `libsoxr` if the input sample rate differs, and maintains a bootstrap buffer until 1 second of audio has accumulated. After tonic locking, frames of 1024 samples (hop 512) are processed sequentially. The processor maintains a slope gate: if the pitch is changing faster than 12 semitones per second, forbidden-note alerts are suppressed to avoid false positives during melodic glides (gamaka ornaments).

The `RagaGrammarValidator` instantiates all five detection layers from the raga database at construction time. For simple ragas (no special phrases), it operates in Mode 1 (membership-only check). For complex ragas with registered prayogas, it operates in Mode 2 (full FSM with phrase tracking). A sticky macro-direction mechanism maintains the last committed melodic direction, updated only when at least two of the last three pairwise steps agree, preventing jitter from ambiguous in-between frames.



### 4.3 Raga Database Design

The raga database stores each raga as a structured record with: arohana (ascending scale), avarohana (descending scale), varja\_arohana (forbidden ascending notes), varja\_avarohana (forbidden descending notes), special\_phrases (characteristic prayoga sequences), characteristic\_phrases (shorter signature motifs), and forbidden\_phrases (explicitly defined illegal note pairs). Vakra forbidden skip pairs are derived automatically at validator initialization by scanning all consecutive triplets in both scales for direction reversals.

### 4.4 Testing and Validation

The test suite covered three levels: unit tests for individual components, integration tests for module pipelines, and end-to-end smoke tests against benchmark audio recordings.

**Table 4.1:** Summary of validation tests

| Test Category          | Description                                          | Outcome |
|------------------------|------------------------------------------------------|---------|
| Swara matching         | Exact ratio match for all 12 swaras (Sa=260 Hz)      | Passed  |
| Octave folding         | Lower, fundamental, upper, and two-octave cases      | Passed  |
| Tolerance boundary     | $\pm 35$ cent accept; $\pm 50$ cent reject           | Passed  |
| Confidence scoring     | 100% at exact; 0% at boundary                        | Passed  |
| Tonic detection        | Three benchmark recordings                           | Passed  |
| Cross-validation       | Pazhani_Shanmuga false positive case                 | Passed  |
| FSM forbidden note     | Ascending/descending varja detection                 | Passed  |
| FSM vakra phrase       | Forbidden skip detection for zigzag ragas            | Passed  |
| Prayoga recognition    | Characteristic phrase suffix match                   | Passed  |
| Debounce logic         | 950 ms trigger, 600 ms clear timing                  | Passed  |
| WebSocket streaming    | Live event dispatch to browser                       | Passed  |
| Octave-aware direction | Pa $\rightarrow$ Sa ascending across octave boundary | Passed  |

## Chapter 5 Results and Discussion

### 5.1 Tonic Detection Accuracy

The Carnatic HPS ensemble was evaluated on three benchmark recordings with ground-truth tonic frequencies measured by an expert. Table 5.1 summarizes the results.

**Table 5.1:** Tonic Sa detection accuracy on benchmark recordings

| Recording        | Duration | GT (Hz) | Detected (Hz) | Error (Hz)  | Error (%)   |
|------------------|----------|---------|---------------|-------------|-------------|
| Enadu_Ullame     | ~5 min   | 124.19  | 123.50        | 0.69        | 0.55        |
| Valli_Kanavan    | ~5 min   | 174.61  | 173.90        | 0.71        | 0.41        |
| Pazhani_Shanmuga | ~3 min   | 170.63  | 174.70        | 4.07        | 2.39        |
| <b>Average</b>   |          |         |               | <b>1.82</b> | <b>1.45</b> |

The first two recordings achieve sub-0.7 Hz error. Pazhani\_Shanmuga is a challenging case where a dominant sung note at 234 Hz achieves the highest Carnatic HPS score, producing a false positive. The cross-validation step detects the disagreement with the standard HPS (which correctly scores 175 Hz) and selects the secondary Carnatic HPS candidate at 174.5 Hz, which is then refined to 174.7 Hz by the fine scan. The resulting 41-cent error (2.39%) is acceptable for automated tonic detection in a practice context.

Simple peak detection (strongest spectral peak without the Carnatic constraint) failed on two of three recordings, finding 263.8 Hz on Valli\_Kanavan (the Pa, not Sa) and 279.9 Hz on Pazhani\_Shanmuga (a sung note). The Carnatic HPS ensemble recovered the correct Sa in all three cases.

### 5.2 Frame Processing and Latency

The system achieved stable per-frame processing performance in all test conditions.

### 5.3 Swara Detection and Grammar Validation

All 12 Carnatic swaras were reliably detected within the  $\pm 35$  cent tolerance across multiple vocal ranges in benchmark tests. Exact-ratio inputs (e.g., Sa at 260 Hz, Pa at 390 Hz) produced confidence scores of 1.0. Inputs at the tolerance boundary (35 cents off) produced confidence scores near 0.0 and were correctly flagged as uncertain. The four-layer gate (RMS, voicing probability, confidence, jump clamp) reduced false positive detections by filtering silence, breath sounds, and rapid pitch transitions.

FSM validation correctly flagged forbidden notes in ascending and descending contexts for all ragas tested, including janya ragas (derived scales with non-trivial varja patterns). Vakra skip detection correctly identified illegal jumps in zigzag scales without false positives

**Table 5.2:** System performance metrics

| Metric                    | Value                                       |
|---------------------------|---------------------------------------------|
| Tonic detection time      | 1.0 s (bootstrap accumulation)              |
| Fine scan processing      | <1 s per file (CPU)                         |
| Frame size                | 1024 samples at 22050 Hz $\approx$ 46 ms    |
| Hop length                | 512 samples $\approx$ 23 ms update interval |
| Per-frame processing time | $\approx$ 35 ms                             |
| End-to-end latency        | $\approx$ 70 ms (bootstrap + 2 frames)      |
| pYIN pitch tracker        | librosa 0.10+                               |
| Tolerance window          | $\pm$ 35 cents                              |
| Alert debounce trigger    | 950 ms                                      |
| Alert clear threshold     | 600 ms                                      |
| Slope gate                | 12 semitones/second (glide suppression)     |

on allowed transitions. Prayoga recognition triggered positive feedback events for all manually entered characteristic phrases.

## 5.4 Comparative Analysis

Table 5.3 summarizes qualitative differences between this work and prior approaches reviewed in Chapter 2.

**Table 5.3:** Qualitative comparison with prior approaches

| Feature                       | This Work    | Deep Classifiers | Statistical Methods |
|-------------------------------|--------------|------------------|---------------------|
| Real-time streaming           | Yes          | No               | No                  |
| Explicit grammar validation   | Yes          | No               | Partial             |
| Microtonal tolerance control  | Yes          | No               | Partial             |
| Explainable per-note feedback | Yes          | Limited          | Limited             |
| Tonic-first design            | Yes          | No               | Partial             |
| Vakra phrase detection        | Yes          | No               | No                  |
| Prayoga recognition           | Yes          | No               | No                  |
| Multi-language feedback       | Yes          | No               | No                  |
| Latency                       | $\sim$ 70 ms | Offline          | Offline             |
| Training data required        | No           | Yes              | No                  |

## 5.5 Model Optimization and Innovation

Key innovations include: (1) the tonic-first design that locks Sa before any grammar check, eliminating octave errors that afflict standard pitch detectors; (2) the weighted Carnatic HPS with Pa-emphasis that distinguishes tambura from sung notes; (3) cross-validation between Carnatic HPS and standard HPS to resolve false positives; (4) octave-aware pairwise direction computation that correctly handles boundary crossings; (5) automatic vakra skip derivation

from raga scales; (6) slope-gated debouncing that suppresses glide false positives while preserving responsiveness to sustained violations; and (7) multi-language teacher-modeled feedback that mirrors how a human instructor would explain a grammar error.

## Chapter 6 Conclusion and Future Work

### 6.1 Conclusion

This thesis presented a complete real-time Carnatic music practice assistant designed around three core technical contributions. First, a Carnatic HPS ensemble for tonic Sa detection that exploits the tambura’s characteristic Sa–Pa frequency pair, achieving an average error of 1.45% across benchmark recordings and correctly recovering from false positives via cross-validation. Second, a tonic-ratio swara quantizer with octave-aware folding, four-layer noise gating, and confidence scoring that maps live pitch to all 12 Carnatic swaras within a  $\pm 35$  cent window at 23 ms update rate. Third, a five-phase multi-layer FSM grammar validator that enforces direction-specific varja rules, detects vakra forbidden skips automatically from the raga scale, recognizes characteristic prayoga phrases, and delivers multi-language teacher-modeled feedback with slope-gated debouncing.

The end-to-end latency of approximately 70 ms and deterministic, rule-based design make the system well-suited for live interactive practice. By placing tonic detection at the center of the architecture rather than treating it as a preprocessing afterthought, the system eliminates the octave ambiguity that undermines standard pitch detectors when applied to Carnatic music. The explicit grammar enforcement at the note level fills a gap left by all existing raga recognition literature, which focuses on offline classification rather than per-note feedback.

### 6.2 Key Learnings

- Tonic-relative processing is essential for reliable swara mapping in Carnatic music; without locking Sa first, standard pitch detectors produce systematic octave errors.
- The tambura Sa–Pa pair is a powerful fingerprint for tonic detection; weighting Pa at 1.0 handles female vocal recordings where the fundamental can be weak.
- Strict cent-based tolerance ( $\pm 35$  cents) improves precision without sacrificing usability; a confidence score gives downstream FSM additional signal quality information.
- Octave-aware pairwise direction computation is necessary for correct grammar enforcement at scale boundaries; naive semitone comparison fails at octave crossings.
- Slope-gated alert debouncing (950 ms trigger, 12 semitone/s glide gate) is essential for suppressing false positives during gamaka ornaments without delaying feedback on genuine sustained violations.
- Automatic vakra skip derivation from raga scales eliminates the need for manual labeling of forbidden pairs, making the system extensible to new ragas with minimal effort.

- Multi-language, parameterized feedback templates allow the system to explain errors the way a teacher would, increasing the pedagogical value beyond binary pass/fail signals.

### 6.3 Future Work

- **Phrase-level session scoring:** Implement a session-level accuracy score that weights forbidden notes and missed prayogas, providing the student with a single practice quality metric.
- **Gamaka and ornament modeling:** Extend swara quantization to recognize continuous pitch ornaments (gamakas) as intentional musical gestures rather than deviations, improving feedback specificity.
- **Automatic raga identification:** Combine swara sequence statistics with prayoga matching to infer the performed raga automatically, enabling raga-selection-free sessions.
- **Microtonal deviation visualization:** Add a time-series plot of cents deviation per swara so students can review their intonation accuracy over a session.
- **Expert annotation dataset:** Record expert Carnatic vocalists across multiple ragas to build a labeled benchmark for quantitative swara detection evaluation.
- **Mobile deployment:** Port the pYIN + FSM pipeline to a mobile backend to enable practice feedback on smartphones without a desktop computer.
- **Adaptive tolerance:** Learn per-student pitch stability profiles and dynamically tighten or relax the tolerance window as accuracy improves.

### 6.4 SDGs Achieved

This project aligns with SDG 4 (Quality Education) by providing accessible, automated, and consistent feedback for Carnatic music students, reducing dependence on scarce expert teacher time. It supports SDG 9 (Industry, Innovation and Infrastructure) by advancing real-time, low-latency audio analytics and demonstrating a reproducible software pipeline for music information retrieval. It contributes to SDG 11 (Sustainable Cities and Communities) by using technology to preserve, document, and promote an intangible cultural heritage, making Carnatic music education more accessible across geographic and economic barriers.

## **Appendices**

## Chapter A Appendix A: Sample Code Snippets

```
1 from tonic_sa_detection import TonicSaDetector
2 from raga_grammar.swara_quantizer import SwaraQuantizer
3
4 # Detect tonic from 1-second bootstrap audio
5 detector = TonicSaDetector(sr=22050)
6 result = detector.ensemble_detection("bootstrap.wav")
7 sa_hz = result["sa_frequency"] # e.g., 260.0 Hz
8
9 # Map a frequency to its nearest Carnatic swara
10 quantizer = SwaraQuantizer(sa_frequency=sa_hz)
11 swara_result = quantizer.to_swara_tonic_band(390.0)
12 # SwaraResult(swara='Pa', octave=0, cents_deviation=0.0, confidence
13 #             =1.0)
14 print(swara_result)
```

**Listing A.1:** Tonic detection and swara mapping

```
1 from raga_grammar.grammar_validator import RagaGrammarValidator
2 from raga_grammar.swara_quantizer import SwaraResult
3
4 validator = RagaGrammarValidator("Bhairavi")
5 sr = SwaraResult(swara="Pa", octave=0, cents_deviation=0.0, confidence
6                 =1.0)
7 event = validator.validate_swara(sr, timestamp_ms=500.0)
8 print(event.error_type) # None (Pa is allowed in Bhairavi)
9
10 sr2 = SwaraResult(swara="Ri2", octave=0, cents_deviation=0.0,
11                  confidence=1.0)
12 event2 = validator.validate_swara(sr2, timestamp_ms=600.0)
13 print(event2.error_type) # ErrorType.FORBIDDEN_NOTE
```

**Listing A.2:** Real-time grammar validation



## Chapter B Appendix B: Example Raga Grammar Rules

```
1 RagaInfo(  
2     name="Bhairavi",  
3     arohana=["Sa", "Ri1", "Ga2", "Ma1", "Pa", "Dha1", "Ni2", "Sa"],  
4     avarohana=["Sa", "Ni2", "Dha1", "Pa", "Ma1", "Ga2", "Ri1", "Sa"],  
5     varja_arohana=[],  
6     varja_avarohana=[],  
7     special_phrases=[],  
8     characteristic_phrases=[  
9         ["Sa", "Ri1", "Sa"],  
10        ["Ma1", "Dha1", "Ma1"],  
11    ],  
12    forbidden_phrases=[]  
13 )
```

**Listing B.1:** Raga database entry (simplified)

```
1 # For a vakra raga like Sankarabharanam variant:  
2 # Arohana: Sa -> Ri -> Ga -> Ri -> Ma -> Pa -> ...  
3 # Zigzag at Ri: (Ga->Ri=-204 cents, Ri->Ma=+316 cents)  
4 # => forbidden skip: [Ga, Ma] derived automatically  
5 from raga_grammar.grammar_validator import _derive_vakra_skip_phrases  
6 skips = _derive_vakra_skip_phrases(["Sa","Ri","Ga","Ri","Ma","Pa","Sa"  
7     ])   
8 print(skips) # [['Ga', 'Ma']]
```

**Listing B.2:** Vakra raga with automatic skip derivation

## Chapter C    Appendix C: Test Checklist

- Tonic detection returns stable Sa for three benchmark recordings within 2.4% error.
- Cross-validation selects secondary candidate when primary is a false positive.
- All 12 swaras match exactly at their just-intonation ratios (confidence = 1.0).
- Inputs  $\pm 35$  cents accepted; inputs  $\pm 50$  cents rejected.
- Octave folding correctly maps lower Sa ( $\frac{1}{2}f_{Sa}$ ) and upper Sa ( $2f_{Sa}$ ).
- Forbidden notes trigger FORBIDDEN\_NOTE in ascending and descending direction.
- Vakra skip pairs detected: illegal  $[a, c]$  jump flagged; allowed  $[a, b, c]$  path passes.
- Characteristic phrase suffix match returns prayoga name on exact tail match.
- Debouncer suppresses alert below 950 ms threshold; fires after 950 ms.
- Alert cleared after 600 ms of clean frames.
- Slope gate (12 st/s) suppresses alert during fast melodic glides.
- WebSocket events received by browser within 100 ms of frame processing.
- Octave-aware direction: Pa (octave 4)  $\rightarrow$  Sa (octave 5) = ASCENDING.
- Session summary returns correct forbidden note count and prayoga count.

## Bibliography

- [1] D. Hebbar and V. Jagtap. “A Comparison of Audio Preprocessing Techniques and Deep Learning Algorithms for Raga Recognition”. In: *Proc. IEEE Int. Conf. Recent Trends Electron., Inf. Commun. Technol. (RTEICT)*. 2023, pp. 342–347.
- [2] “Deep Learning-Based Classification of Indian Classical Music Based on Raga”. In: *IEEE Trans. Multimedia* 24 (2022), pp. 1845–1856.
- [3] A. Krishnaswamy. *Application of Pitch Tracking to South Indian Classical Music*. Tech. Rep. Stanford University CCRMA, 2018.
- [4] “Melody Extraction from Polyphonic Music by Deep Learning”. In: *Proc. ISMIR*. 2020, pp. 621–627.
- [5] “Singing Voice Separation and Pitch Extraction from Monaural Polyphonic Audio Music via DNN and Adaptive Pitch Tracking”. In: *Proc. ICASSP*. 2019, pp. 701–705.
- [6] D. Peri. “Applying Natural Language Processing and Deep Learning Techniques for Raga Recognition in Indian Classical Music”. MA thesis. Virginia Tech, 2023.
- [7] S. Natesan and H. Beigi. “Carnatic Raga Identification System using Rigorous Time-Delay Neural Network”. In: *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2020, pp. 3162–3166.
- [8] “Carnatic Classical Raga Identification Using Stacked Ensemble Learning Model”. In: *arXiv preprint* (2022).
- [9] “Explainable Deep Learning Analysis for Raga Identification in Indian Art Music”. In: *Proc. Int. Soc. Music Inf. Retr. Conf. (ISMIR)*. 2023.
- [10] “Phrase based Raga Recognition - Pitch Distribution Methods”. In: *Int. J. Comput. Appl.* 132.7 (2015).
- [11] “Raga Space Visualization using Dimensionality Reduction Techniques”. In: *Proc. Natl. Conf. Commun. (NCC)*. 2022.
- [12] “Vector Space Modelling for Microtonal Music Analysis”. In: *Pattern Recognit. Lett.* 145 (2021), pp. 89–95.
- [13] “Self-Extracting Feature Maps for Music Audio Analysis”. In: *IEEE Signal Process. Lett.* 28 (2021), pp. 1452–1456.
- [14] “Comparative Study of the Working of Various AI Classifiers for Identifying Ragas of Indian Classical Music”. In: *Proc. Int. Conf. Comput. Intell. Data Sci.* 2021.
- [15] “Investigation into the Identification of Ragas: A Detailed Analysis”. In: *J. Acoust. Soc. India* 48.3 (2021).